MERN STACK

E-Commerce Web App

Roman Urdu Mein — Zero Se Hero Tak

CHAPTER 5 | SECURITY | JWT + BCRYPT | ROMAN URDU

CHAPTER 5

Authentication & Authorization

JWT aur bcrypt se app ko secure banao — asli security seekho!

Is Chapter Mein Aap Seekhenge:

- Authentication vs Authorization — dono mein farq
- bcrypt — password hashing ka complete concept
- JWT — JSON Web Token kya hai aur kaise kaam karta hai
- User Register — password hash karke safely store karna
- User Login — password verify, JWT token generate karna
- protect Middleware — routes ko secure karna
- admin Middleware — role-based access control
- Token refresh aur logout ka concept
- Secured endpoints Thunder Client se test karna

■ Chapters 1–4 Recap	Express, MongoDB, Mongoose models, MVC controllers, routes — yeh sab aana chahiye. Chapter 5 mein hum in routes ko LOCK karte hain!

TABLE OF CONTENTS

8.0 Routes Final Update

- ⬜ 8.1 productRoutes — admin protected
- ⬜ 8.2 userRoutes — protect applied
- ⬜ 8.3 orderRoutes — user protected

9.0 Thunder Client se Auth Testing

- ⬜ 9.1 Register test
- ⬜ 9.2 Login test — token save karo
- ⬜ 9.3 Protected route test — Bearer token
- ⬜ 9.4 Admin route test

10.0 Summary & Practice

- ⬜ 10.1 Chapter recap
- ⬜ 10.2 Practice exercises
- ⬜ 10.3 Chapter 6 preview

SECTION 1 — Authentication vs Authorization

1.0 — Yeh Do Words Alag Hain!

Bahut log yeh dono confuse karte hain — lekin yeh bilkul alag cheezein hain. Ek simple example se samjhate hain:

■ OFFICE BUILDING	■ SHOPEASE APP
Authentication Security guard ne ID card dekha — 'Tum kaun ho?' — pehchaan confirm ki. Andar aane diya.	**Authentication** User ne email+password diya — server ne verify kiya — 'Haan yeh valid user hai' — JWT token diya.
Authorization Andar jaake CEO ke room mein jaana chaha — guard ne roka: 'Teri permission nahi!'	**Authorization** User ne /admin/products delete karna chaha — server ne roka: 'Sirf admin kar sakta hai!'

1.1 — Ek Line Mein Farq

■ Authentication	■ Authorization
TUM KAUN HO? Login karo — identity prove karo	**TUMHE KYA KARNE KI IJAZAT HAI?** Role check karo — permission verify karo
JWT Token se hota hai	Role field se hota hai (user/admin)

■ TIP

ShopEase mein: Authentication = 'Kya user logged in hai?' — JWT se. Authorization = 'Kya woh admin hai?' — role field se. Dono alag middlewares handle karte hain!

SECTION 2 — bcrypt: Password Hashing

2.0 — Plain Text Password Kyun Dangerous Hai?

Socho agar hum database mein password aise store karein: **password: 'ali123'** — agar koi hacker database hack kare toh saare passwords seedhe mil jaate hain! Millions of users affected ho sakte hain. Yeh ek bahut badi security mistake hai.

Scenario	Plain Text	Hashed (bcrypt)
DB hack ho gaya	Hacker ko seedha password milta	Hacker ko hash milta — useless!
Employee DB access kare	Woh user ke passwords dekh sakta	Sirf hashes — kuch nahi samjhe
Same password users	Database mein same string	Har hash alag — salting ki wajah se
Password crack karna	Seedha pata chal jaata	Practically impossible — bohat slow

2.1 — Hashing Kya Hai?

Hashing ek **one-way process** hai — password ko ek scrambled string mein convert karo. Wapas original password nahi mil sakta! Sirf **compare** kar sakte hain:

```
// Plain text password
Input:  'ali123'

// bcrypt hash — completely alag!
Output: '$2b$12$LQv3c1yqBWVHxkd0LHAkCOYz6TtxMQJqhN8/LewdBPj/VmzXCiAWq'

// Dobara hash karo — PHIR BHI alag! (salting ki wajah se)
Output: '$2b$12$8K1p/a0dR1xqM0DTvm4F.uZ/VJZKQz7KvqF3L.5pByqA6B0v2yKy2'

// Lekin compare karo — dono 'ali123' se match karte hain!
bcrypt.compare('ali123', hash1)  // true
bcrypt.compare('ali123', hash2)  // true
bcrypt.compare('wrong', hash1)   // false
```

2.2 — Salt Rounds Kya Hai?

Salt ek random string hai jo password mein add hoti hai hashing se pehle — isliye same password ka hash har baar alag hota hai. **Salt rounds** batata hai hashing kitni baar repeat hogi — zyada rounds = zyada secure = lekin thoda slow:

Salt Rounds	Speed	Security	Recommended?
8	Bohot fast	Kam secure	■ Avoid
10	Fast	Good	■ Default — most apps
12	Medium	Very Good	■ Recommended — ShopEase mein yeh use karein
14	Slow	Excellent	■ Banks/High security apps
16+	Bahut Slow	Overkill	■ UX kharab hoti hai

2.3 — bcrypt Install aur Basic Usage

```
# bcryptjs install karo (pure JS — better compatibility)
npm install bcryptjs
```

```
const bcrypt = require('bcryptjs');

// ■■ Password Hash Karna ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
const password   = 'ali123';
const saltRounds = 12;

// Method 1: Auto salt (recommended)
const hashedPassword = await bcrypt.hash(password, saltRounds);
// '$2b$12$...' — yeh store karo DB mein

// ■■ Password Compare Karna ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
const isMatch = await bcrypt.compare('ali123', hashedPassword);
console.log(isMatch);  // true

const isMatch2 = await bcrypt.compare('wrong', hashedPassword);
console.log(isMatch2); // false
```

■ TIP

Hamesha bcryptjs use karo — bcrypt nahi. bcryptjs pure JavaScript hai, kisi bhi system pe easily install hota hai. bcrypt C++ bindings use karta hai — sometimes installation issues aate hain.

SECTION 3 — JWT: JSON Web Token

3.0 — JWT Kya Hai?

JWT ek **token** hai — ek string — jo server user ko deta hai login ke baad. Agali baar jab user koi request kare, yeh token saath bheje — server samajh jaata hai 'yeh wahi user hai jisne login kiya tha.' Jaise ek concert ka wristband — ek baar laga lo, baar baar ticket nahi dikhana!

3.1 — JWT Ki 3 Parts

JWT ek dot (.) se separated 3 parts hoti hai:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY0ZjhhM2IyYzFkMmUzZjQiLCJpYXQiOjE2OTQwMDAwMDAsImV4cCI6MTY5NjU5MjAwMH0.8K1p_a0dR1xqM0DTvm4F

# Part 1: HEADER  (algorithm info)
# eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
# Decode karo: { alg: 'HS256', typ: 'JWT' }

# Part 2: PAYLOAD (actual data — public!)
# eyJpZCI6IjY0ZjhhM2IyYzFkMmUzZjQiLCJpYXQiOjE2OTQwMDAwMDAsImV4cCI6MTY5NjU5MjAwMH0
# Decode karo: { id: '64f8a3b2c1d2e3f4', iat: 1694000000, exp: 1696592000 }

# Part 3: SIGNATURE (tamper-proof — secret se bani)
# 8K1p_a0dR1xqM0DTvm4F
# Agar koi payload change kare — signature match nahi karega — reject!
```

■ YAAD RAHO

PAYLOAD public hoti hai — koi bhi decode kar sakta hai! Isliye KABHI bhi sensitive data (password, credit card) payload mein mat rakho. Sirf ID aur role rakho. Signature tampering se bachata hai — content se nahi!

3.2 — JWT Ka Complete Flow

1	User	Login request bhejta hai: email + password
2	Server	Password verify karta hai bcrypt se

3	Server	JWT token generate karta hai: jwt.sign({ id }, secret, { expiresIn })
4	Server	Token response mein bhejta hai
5	Client	Token save karta hai (localStorage ya cookie mein)
6	Client	Agli request mein: Authorization: Bearer header add karta hai
7	Server	Token verify karta hai: jwt.verify(token, secret)
8	Server	Payload se user ID nikalta hai — DB se user fetch karta hai
9	Server	Request process karta hai — response bhejta hai

3.3 — JWT vs Sessions — Kyun JWT Better Hai?

Feature	Sessions (purana tarika)	JWT (modern tarika) ■
Storage	Server mein session store hota	Sirf client ke paas — server stateless
Scale karna	Mushkil — sab servers ka ek DB	Asaan — koi bhi server verify kar sakta
Mobile apps	Cookies ka masla	Header mein bhejo — everywhere works
Microservices	Complicated	Alag services bhi verify kar sakti hain
Logout	Server se session delete karo	Token expiry par rely karo
Performance	Har request pe DB hit	Sirf verify — no DB needed usually

```
# JWT install karo
npm install jsonwebtoken
```

SECTION 4 — User Model Update

4.0 — User Model Mein 3 Cheezein Add Karni Hain

1. pre('save') hook — save se pehle password auto-hash karo
2. comparePassword() method — login mein password verify karo
3. generateToken() method — JWT token banana

4.1 — Complete Updated User Model

■ models/User.js — Final Version

```
const mongoose = require('mongoose');
const bcrypt   = require('bcryptjs');
const jwt      = require('jsonwebtoken');

const userSchema = new mongoose.Schema({
  name:  { type: String, required: [true,'Naam zaroori hai!'], trim: true
 },
  email: {
    type:     String,
    required: [true,'Email zaroori hai!'],
    unique:   true,
    lowercase:true,
    match:    [/^\S+@\S+\.\S+$/, 'Valid email daalo!'],
  },
  password: {
    type:     String,
    required: [true,'Password zaroori hai!'],
    minLength:[6,'Password 6+ characters ka hona chahiye!'],
    select:   false,  // queries mein auto nahi aayega
  },
  role:    { type: String, enum: ['user','admin'], default: 'user' },
  avatar:  { type: String, default: 'default.jpg' },
  address: { street:String, city:String, state:String, zipCode:String },
  isActive:{ type: Boolean, default: true },
```

```
}, { timestamps: true });

// ■■ 1. pre('save') Hook — Password Auto Hash ■■■■■■■■■■
// Yeh function tab chalta hai jab bhi user.save() call ho
userSchema.pre('save', async function(next) {
  // Agar password change nahi hua toh hash mat karo
  // (naam ya email update karte waqt bhi save() chalta hai)
  if (!this.isModified('password')) return next();

  // Hash karo — 12 salt rounds
  this.password = await bcrypt.hash(this.password, 12);
  next();
});

// ■■ 2. comparePassword() Method ■■■■■■■■■■■■■■■■■■■■■■
// Login ke waqt use hoga — entered password ko hash se compare karo
userSchema.methods.comparePassword = async function(enteredPassword) {
  // this.password = DB mein stored hash
  return await bcrypt.compare(enteredPassword, this.password);
};

// ■■ 3. generateToken() Method ■■■■■■■■■■■■■■■■■■■■■■■■
// Login/Register ke baad JWT token banana
userSchema.methods.generateToken = function() {
  return jwt.sign(
    { id: this._id },                    // Payload: sirf ID
    process.env.JWT_SECRET,               // Secret key .env se
    { expiresIn: process.env.JWT_EXPIRE || '30d' }  // Expiry
  );
};

module.exports = mongoose.model('User', userSchema);
```

`this.isModified('password')`	Mongoose method — check karta hai ke kya yeh specific field change hui hai is save() call mein. Agar nahi badli toh dobara hash karna galat hoga.

`userSchema.methods.X`	Instance method banana — yeh method har User document (object) pe available hoga. jaise user.comparePassword() ya user.generateToken().
`bcrypt.compare(plain, hash)`	Plain text aur hash ko compare karta hai — true ya false return karta hai. Kabhi bhi manually hash na banao compare ke liye — yeh function karo.
`jwt.sign(payload, secret, opt)`	JWT token generate karta hai. Payload = data jo token mein jayega. Secret = signing key. Options = expiry wagera.

SECTION 5 — Register & Login Complete Flow

5.0 — Register Controller — Final Version

■ controllers/userController.js

```
const asyncHandler = require('../middleware/asyncHandler');
const User         = require('../models/User');

// ■■ Helper: Token response bhejna ■■■■■■■■■■■■■■■■■■■■■■
const sendTokenResponse = (user, statusCode, res) => {
  const token = user.generateToken();

  res.status(statusCode).json({
    success: true,
    token,
    data: {
      _id:    user._id,
      name:   user.name,
      email:  user.email,
      role:   user.role,
      avatar: user.avatar,
    },
  });
};

// ■■ @route  POST /api/users/register ■■■■■■■■■■■■■■■■■■
// ■■ @access Public
const registerUser = asyncHandler(async (req, res) => {
  const { name, email, password } = req.body;

  // 1. Validation — sab fields hain?
  if (!name || !email || !password) {
    res.status(400);
    throw new Error('Naam, email aur password — teeno zaroori hain!');
```

```
  }

  // 2. Email pehle se registered?
  const userExists = await User.findOne({ email: email.toLowerCase() });
  if (userExists) {
    res.status(400);
    throw new Error('Yeh email pehle se registered hai!');
  }

  // 3. User banao — password pre('save') hook se hash hoga
  const user = await User.create({ name, email, password });

  // 4. Token ke saath response bhejo
  sendTokenResponse(user, 201, res);
});

// ■■ @route  POST /api/users/login ■■■■■■■■■■■■■■■■■■■■■■
// ■■ @access Public
const loginUser = asyncHandler(async (req, res) => {
  const { email, password } = req.body;

  // 1. Email aur password dono hain?
  if (!email || !password) {
    res.status(400);
    throw new Error('Email aur password dono daalo!');
  }

  // 2. User dhundho — password bhi select karo (select:false tha)
  const user = await User.findOne({ email: email.toLowerCase() }).select(
'+password');

  // 3. User exists? Password match?
  // Dono checks ek saath — security ke liye (timing attack se bacho)
  if (!user || !(await user.comparePassword(password))) {
    res.status(401);
    throw new Error('Email ya password galat hai!');
  }

  // 4. Sab theek — token bhejo
  sendTokenResponse(user, 200, res);
```

```
});

// ■■ @route  GET /api/users/profile ■■■■■■■■■■■■■■■■■■■■
// ■■ @access Private (protect middleware lagega)
const getUserProfile = asyncHandler(async (req, res) => {
  // req.user protect middleware ne set kiya hoga
  const user = await User.findById(req.user._id);
  res.status(200).json({ success: true, data: user });
});

// ■■ @route  PUT /api/users/profile ■■■■■■■■■■■■■■■■■■■■
// ■■ @access Private
const updateUserProfile = asyncHandler(async (req, res) => {
  const user = await User.findById(req.user._id);
  if (!user) { res.status(404); throw new Error('User nahi mila!'); }

  // Sirf woh fields update karo jo bheji gayi hain
  if (req.body.name)     user.name     = req.body.name;
  if (req.body.email)    user.email    = req.body.email;
  if (req.body.password) user.password = req.body.password;  // pre hook
hash karega
  if (req.body.address)  user.address  = req.body.address;

  const updated = await user.save();  // pre('save') chalta hai
  sendTokenResponse(updated, 200, res);
});

module.exports = { registerUser, loginUser, getUserProfile, updateUserPro
file };
```

■ TIP

Login mein 'Email ya password galat hai!' — dono cases mein same message kyon? Security ke liye! Agar alag messages hote ('Email nahi mila' ya 'Password galat') toh hacker ko pata chal jaata ke email valid hai ya nahi. Same message se woh kuch nahi samajh sakta!

SECTION 6 — protect Middleware

6.0 — protect Middleware Kya Karta Hai?

protect middleware har protected route ke beech mein ek **security guard** ki tarah kaam karta hai. Jab bhi koi user /api/users/profile ya /api/orders jaana chahta hai — pehle is guard se guzarna padta hai:

Step 1	**Token dhundho**	Authorization header mein 'Bearer ' format mein token aata hai — extract karo
Step 2	**Token verify karo**	jwt.verify() se check karo — valid hai? Expire nahi hua?
Step 3	**User dhundho**	Token payload mein ID hai — DB se user fetch karo
Step 4	**req.user set karo**	User object req.user pe attach karo — agle middleware/controller use kar sakein
Step 5	**next() call karo**	Sab theek — route handler ko jaane do

6.1 — protect Middleware Code

■ middleware/authMiddleware.js

```
const jwt        = require('jsonwebtoken');
const asyncHandler = require('./asyncHandler');
const User       = require('../models/User');

// ■■ protect — Login check karo ■■■■■■■■■■■■■■■■■■■■■■■■■
const protect = asyncHandler(async (req, res, next) => {
  let token;

  // Token Authorization header mein hona chahiye
  // Format: 'Bearer eyJhbGciOi...'
  if (req.headers.authorization &&
      req.headers.authorization.startsWith('Bearer')) {
    // 'Bearer token123' se sirf 'token123' lo
```

```
    token = req.headers.authorization.split(' ')[1];
  }

  // Token nahi mila?
  if (!token) {
    res.status(401);
    throw new Error('Login zaroori hai! Pehle login karo.');
  }

  // Token verify karo
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  // decoded = { id: '64f8a3b2...', iat: ..., exp: ... }

  // DB se user fetch karo — abhi bhi active hai?
  const user = await User.findById(decoded.id);
  if (!user) {
    res.status(401);
    throw new Error('Is token ka user ab exist nahi karta!');
  }

  if (!user.isActive) {
    res.status(401);
    throw new Error('Account deactivate ho chuka hai!');
  }

  // User ko request object par attach karo
  // Ab koi bhi agle middleware/controller req.user use kar sakta hai
  req.user = user;
  next();
});

module.exports = { protect };
```

`req.headers.authorization`	HTTP request headers mein se Authorization header lo. Client ne 'Bearer ' bheja hoga.
`.split(' ')[1]`	'Bearer token123' string ko space par split karo — array ['Bearer','token123'] banta hai — index [1] se token lo.

`jwt.verify(token, secret)`	Token verify karta hai. Agar valid hai toh decoded payload return karta hai. Agar invalid ya expired — error throw karta hai jo asyncHandler pakad lega.
`req.user = user`	Yeh line bahut important hai! User object ko request par attach karo taki agle controllers mein req.user se access ho sake.

SECTION 7 — admin Middleware

7.0 — Role-Based Access Control

protect middleware sirf check karta hai: 'logged in hai ya nahi?' Lekin kuch routes sirf admin ke liye hain — jaise product delete, all orders dekhna, user ko ban karna. In ke liye **admin middleware** chahiye — protect ke BAAD lagta hai:

```
// authMiddleware.js mein add karo (protect ke baad)

// ■■ admin — Role check karo ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
const admin = (req, res, next) => {
  // protect middleware pehle chal chuka hai
  // req.user available hai
  if (req.user && req.user.role === 'admin') {
    next();  // Admin hai — aage jao
  } else {
    res.status(403);
    throw new Error('Admin access required! Tumhare paas permission nahi.
');
  }
};

// ■■ seller — Seller role check (future use) ■■■■■■■■■■■
const seller = (req, res, next) => {
  if (req.user && (req.user.role === 'seller' || req.user.role === 'admin
')) {
    next();
  } else {
    res.status(403);
    throw new Error('Seller access required!');
  }
};

module.exports = { protect, admin, seller };
```

7.1 — protect + admin Milke Kaise Lagate Hain

Route par middleware chain karo — order important hai! protect pehle, phir admin:

```
// routes/productRoutes.js
const { protect, admin } = require('../middleware/authMiddleware');

// Public — koi bhi dekh sakta
router.get('/',    getProducts);
router.get('/:id', getProductById);

// User — sirf logged in user
router.post('/:id/reviews', protect, createReview);
//                          ↑ pehle login check

// Admin — pehle login, phir admin check
router.post('/',      protect, admin, createProduct);
router.put('/:id',    protect, admin, updateProduct);
router.delete('/:id', protect, admin, deleteProduct);
//                    ↑ login  ↑ admin

// Agar protect fail hoga — admin tak jaayega hi nahi
// Agar admin fail hoga — controller tak nahi pahonchega
```

■ TIP

403 aur 401 mein farq: 401 Unauthorized = login nahi kiya. 403 Forbidden = login kiya lekin permission nahi. Yeh alag status codes hain — theek theek use karo!

7.2 — Kisi User Ko Admin Banana

Database mein seedha update karo — ya ek temporary admin route banao:

```
// Option 1: MongoDB Atlas mein directly field update karo
// Collections -> users -> document -> role field -> 'admin'

// Option 2: Mongoose se update karo (terminal script)
// scripts/makeAdmin.js banao:
require('dotenv').config();
const mongoose = require('mongoose');
const User     = require('./models/User');

const makeAdmin = async () => {
  await mongoose.connect(process.env.MONGO_URI);
```

```
  const user = await User.findOneAndUpdate(
    { email: 'admin@shopease.com' },
    { role: 'admin' },
    { new: true }
  );
  console.log('Admin bana diya:', user.name, '—', user.role);
  process.exit(0);
};
makeAdmin();

// Chalao: node scripts/makeAdmin.js
```

SECTION 8 — Routes Final Update

8.0 — Ab Saari Routes Secure Hain!

Chapter 4 mein routes mein auth nahi tha — koi bhi product delete kar sakta tha! Ab protect aur admin middleware add karte hain:

8.1 — productRoutes.js — Final

■ routes/productRoutes.js

```
const express  = require('express');
const router   = express.Router();
const { protect, admin } = require('../middleware/authMiddleware');
const {
  getProducts, getProductById, createProduct,
  updateProduct, deleteProduct, createReview
} = require('../controllers/productController');

// Public routes
router.get('/',    getProducts);
router.get('/:id', getProductById);

// User routes — login zaroori
router.post('/:id/reviews', protect, createReview);

// Admin routes — login + admin role zaroori
router.post('/',      protect, admin, createProduct);
router.put('/:id',    protect, admin, updateProduct);
router.delete('/:id', protect, admin, deleteProduct);

module.exports = router;
```

8.2 — userRoutes.js — Final

■ routes/userRoutes.js

```
const express  = require('express');
const router   = express.Router();
```

```
const { protect, admin } = require('../middleware/authMiddleware');
const {
  registerUser, loginUser, getUserProfile, updateUserProfile
} = require('../controllers/userController');

// Public routes — koi bhi
router.post('/register', registerUser);
router.post('/login',    loginUser);

// Private routes — login zaroori
router.get('/profile',  protect, getUserProfile);
router.put('/profile',  protect, updateUserProfile);

module.exports = router;
```

8.3 — orderRoutes.js — Final

■ routes/orderRoutes.js

```
const express  = require('express');
const router   = express.Router();
const { protect, admin } = require('../middleware/authMiddleware');
const {
  createOrder, getMyOrders, getOrderById,
  updateOrderStatus, getAllOrders
} = require('../controllers/orderController');

// User routes — login zaroori
router.post('/',           protect, createOrder);
router.get('/myorders',    protect, getMyOrders);
router.get('/:id',         protect, getOrderById);

// Admin routes
router.get('/',            protect, admin, getAllOrders);
router.put('/:id/status',  protect, admin, updateOrderStatus);

module.exports = router;
```

■ SAHI KIYA

Ab ShopEase ka backend completely secure hai! Public routes khule hain. User routes login ke peeche hain. Admin routes double lock hain — login + admin role. Yahi production-ready security pattern hai!

SECTION 9 — Thunder Client se Auth Testing

9.0 — Step by Step Auth Test

Is order mein test karo — yeh exact sequence follow karo:

9.1 — Test 1: User Register

```
// Method: POST
// URL: http://localhost:5000/api/users/register
// Headers: Content-Type: application/json
// Body (JSON):
{
  "name":     "Ali Khan",
  "email":    "ali@shopease.com",
  "password": "ali12345"
}

// Expected Response (201):
{
  "success": true,
  "token":   "eyJhbGci...",   // YEH TOKEN COPY KARO!
  "data": { "_id": "...", "name": "Ali Khan", "email": "...", "role": "us
er" }
}
```

9.2 — Test 2: User Login

```
// Method: POST
// URL: http://localhost:5000/api/users/login
// Body (JSON):
{
  "email":    "ali@shopease.com",
  "password": "ali12345"
}
```

```
// Expected Response (200):
{
  "success": true,
  "token":   "eyJhbGci...",   // Naya token — yeh bhi copy karo
  "data": { "_id": "...", "name": "Ali Khan", "role": "user" }
}

// Galat password test:
// Expected Response (401):
// { success: false, message: 'Email ya password galat hai!' }
```

9.3 — Test 3: Protected Route — Bearer Token

Ab profile route test karo — yeh login ke bina nahi khulega:

```
// Test A: Bina token ke (fail hona chahiye)
// Method: GET
// URL: http://localhost:5000/api/users/profile
// Headers: koi Authorization nahi
// Expected: 401 — 'Login zaroori hai!'

// Test B: Token ke saath (success hona chahiye)
// Method: GET
// URL: http://localhost:5000/api/users/profile
// Headers:
//   Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
//                  ↑ 'Bearer ' phir token paste karo
// Expected: 200 — user profile data

// Thunder Client mein:
// Headers tab -> Add Header
// Key:   Authorization
// Value: Bearer <yahan token paste karo>
```

9.4 — Test 4: Admin Route

```
// Pehle apne aap ko admin banao:
// node scripts/makeAdmin.js
```

```
// Ya MongoDB Atlas mein role field 'admin' kar do

// Phir dobara LOGIN karo — naya token lo (admin role wala)

// Test: Product delete (admin only)
// Method: DELETE
// URL: http://localhost:5000/api/products/<kisi_product_ka_id>
// Headers: Authorization: Bearer <admin_token>
// Expected: 200 — 'Product delete ho gaya!'

// Regular user token se try karo (fail hona chahiye):
// Expected: 403 — 'Admin access required!'
```

■ TIP

Thunder Client Tip: Environment Variables use karo! Thunder Client mein Env tab mein 'token' variable set karo — phir har header mein {{token}} likh do. Ek jagah update karo — sab jagah update ho jaata hai!

SECTION 10 — Summary, Practice & Next Steps

10.0 — Chapter 5 Ka Recap

Authentication	Kaun ho tum? — JWT token se verify karna
Authorization	Kya kar sakte ho? — Role se permission check
bcryptjs	Password hashing — one-way, safe, salted
Salt Rounds	12 recommended — zyada = zyada secure but slow
JWT	JSON Web Token — stateless auth, 3 parts: header.payload.signature
jwt.sign()	Token banana — payload + secret + expiry
jwt.verify()	Token verify karna — valid? expired? tampered?
pre('save') Hook	Password auto-hash — save se pehle automatically
comparePassword()	Instance method — bcrypt.compare() wrapper
generateToken()	Instance method — jwt.sign() wrapper
sendTokenResponse()	Helper — token + user data ek format mein bhejna
protect MW	Token extract -> verify -> user fetch -> req.user set
admin MW	req.user.role === 'admin' check — 403 if not
401 vs 403	401 = not logged in | 403 = logged in but no permission
Middleware Chain	protect, admin — order important hai — pehle protect!

10.1 — Practice Exercises

Easy Level

1. bcryptjs aur jsonwebtoken install karo
2. User model mein pre('save'), comparePassword(), generateToken() add karo
3. Register route test karo — response mein token aana chahiye
4. Login test karo — sahi aur galat password dono

■ Medium Level

1. protect middleware banao aur test karo
2. GET /api/users/profile — bina token 401, token se 200
3. admin middleware banao
4. Apne aap ko admin banao — makeAdmin script likho

■ Challenge Level

1. Saari routes par correct middleware lagao
2. Regular user se admin route try karo — 403 aana chahiye
3. Admin user se admin route — success aana chahiye
4. Profile update karo — password change karo — dobara login karo
5. Galat token bhejo — error message check karo
6. Expired token simulate karo — JWT_EXPIRE='5s' set karo, 5 sec baad try karo

10.2 — Chapter 6 Ka Preview

React.js Basics — Frontend Shuru Karo!

- ■ React kya hai — library vs framework
- ■ Create React App — project setup karna
- ■ JSX — JavaScript + HTML ka combo
- ■ Components — reusable UI pieces banana
- ■ Props — parent se child ko data bhejna
- ■ useState Hook — component ka apna data
- ■ useEffect Hook — side effects handle karna
- ■ ShopEase frontend structure banana

Waah! Backend Complete Ho Gaya! ■■

Chapters 1-5 mein tumne poora MERN backend banaya — Express server, MongoDB database, MVC architecture, aur ab complete authentication. Yeh sab woh kaam hai jo real companies ke senior developers karte hain! **Chapter 6 se ab React shuru hoga — frontend ki duniya mein khush aamdeed!**